

YallaMarket

Software Requirements Specification (SRS)

Prepared for Software Engineering Course

Project	YallaMarket
Document Title	Software Requirements Specification
Version	V2.0
Date	April 2026
Methodology	Agile / Scrum

Document summary: This SRS defines the functional and non-functional requirements for YallaMarket, a bilingual digital marketplace for game cards, subscriptions, and other digital balance products.

Contents

1. Introduction	3
1.1 Purpose	3
1.2 Product Overview	3
1.3 Scope	3
1.4 Definitions and Acronyms	3
1.5 References.....	4
2. Overall Description	4
2.1 Product Perspective	4
2.2 User Classes	4
2.3 Operating Environment.....	4
2.4 Design and Implementation Goals	4
3. System Architecture	4
3.1 Architectural Principles	5
4. Functional Requirements	5
5. External Interface Requirements	6
5.1 User Interface	6
5.2 Software Interfaces	6
5.3 Hardware and Communication Interfaces	6
6. Non-Functional Requirements	7
7. Use Cases	7
8. Data Requirements	8
8.1 Core Entities	8
9. Constraints, Assumptions, and Dependencies.....	9
9.1 Technical Constraints.....	9
9.2 Assumptions	9
9.3 Dependencies	10
10. Agile Development Plan	10
10.1 Team Roles	10
10.2 Sprint Plan	10
10.3 Current Jira Direction	11
11. Appendices	11
11.1 Planned API Summary (Future Phase).....	11
11.2 Revision History.....	11

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the requirements for YallaMarket, a web-based digital marketplace focused on selling game cards, subscriptions, and electronic balances. The document is intended for the project team, instructors, reviewers, and future maintainers.

1.2 Product Overview

YallaMarket provides a storefront experience similar to a modern e-commerce platform, with a bilingual Arabic/English interface and category-driven browsing flow. Users can navigate from the homepage to a category page, open a product details page, add items to a shopping cart, and complete a checkout form. The current phase emphasizes a polished frontend, while backend integration, authentication, and payment processing are planned future phases.

1.3 Scope

- Bilingual storefront interface with Arabic (RTL) and English (LTR) support.
- Homepage with categories and promotional sections.
- Category page with product listing and in-page search/filter behavior.
- Product details page with description, price, stock status, and related products.
- Shopping cart with quantity control, subtotal, delivery fee, and total.
- Checkout page UI prepared for future backend and payment integration.
- Project structure prepared for future Express + SQLite integration.

1.4 Definitions and Acronyms

Term	Meaning
SRS	Software Requirements Specification
RTL	Right-to-left layout used for Arabic content
LTR	Left-to-right layout used for English content
UI	User Interface
UX	User Experience
API	Application Programming Interface
JWT	JSON Web Token, planned for future authentication
SQLite	Lightweight relational database planned for backend persistence
Mock Data	Temporary static data used during frontend development

1.5 References

- IEEE-style SRS structure and best practices for requirements documentation.
- Biddimarket.ps as inspiration for navigation flow and marketplace interaction.
- React, Vite, Tailwind CSS, and React Router official documentation.
- Project artifacts including Jira backlog, sprint plan, and UI prototypes.

2. Overall Description

2.1 Product Perspective

YallaMarket is designed as a modular web application. In the current phase, the system operates as a frontend-first solution using reusable components, mock data, and centralized state management for the shopping cart. In later phases, the same frontend will connect to a backend service built with Express and SQLite, enabling persistence, user accounts, order processing, and administration.

2.2 User Classes

User Class	Description	Permissions in Current Scope
Guest	Unauthenticated visitor browsing the storefront.	Browse categories, products, cart, and checkout UI.
Registered User (future)	Customer with an account.	Will place orders, manage profile, and view order history.
Admin (future)	Store manager or operator.	Will manage products, categories, orders, and dashboard data.

2.3 Operating Environment

- Web browsers: latest Chrome, Edge, Firefox, and Safari.
- Responsive layout targeting mobile, tablet, and desktop screen sizes.
- Frontend stack: React + Vite + Tailwind CSS.
- Current data source: mock service layer.
- Planned backend stack: Express + SQLite.

2.4 Design and Implementation Goals

- Preserve a clean and modern store layout while keeping components reusable.
- Avoid unnecessary coupling between UI and data source implementation.
- Prepare the project for future backend integration with minimal refactoring.
- Support both Arabic and English content without duplicating layouts.

3. System Architecture

The solution follows a presentation-driven architecture during the current phase and evolves toward a standard three-layer architecture in future phases.

Layer	Current Implementation	Planned Evolution	Responsibility
Presentation	React + Tailwind + React Router	Same stack	User interface, routing, interaction, language-aware layout
Application	Frontend services + Context	Express.js backend	Business logic, API endpoints, validation, order workflows
Data	Mock data files	SQLite database	Product, category, order, and user persistence

3.1 Architectural Principles

- Reusable components should be used across related pages.
- State that is shared across pages should be centralized.
- Services should abstract access to product and category data.
- UI should remain stable even when backend persistence is introduced later.

4. Functional Requirements

The following functional requirements define the expected system behavior for the storefront and planned future phases.

ID	Requirement	Description	Priority
FR-01	Browse categories	The system shall display storefront categories on the homepage.	High
FR-02	View category products	The system shall display products that belong to the selected category.	High
FR-03	Search within category	The system shall allow filtering products by name within a category page.	Medium
FR-04	View product details	The system shall show name, description, price, image placeholder, stock status, and related products.	High
FR-05	Add to cart	The system shall allow adding a product to the cart from a product card or product page.	High
FR-06	Update quantity	The system shall allow increasing and decreasing quantity for cart items.	High
FR-07	Remove cart item	The system shall allow removing an item from the cart.	High
FR-08	Persist cart	The system shall preserve cart contents in localStorage between sessions.	High
FR-09	View cart summary	The cart page shall display subtotal, delivery fee, and total.	High

ID	Requirement	Description	Priority
FR-10	Checkout form	The system shall provide a checkout UI with customer information, address, notes, payment method, and order summary.	High
FR-11	Language switch	The system shall support switching between Arabic and English interfaces.	High
FR-12	Display feedback	The system shall show user feedback for actions such as add-to-cart or invalid forms.	Medium
FR-13	Authentication (future)	The system shall support registration and login in a future phase.	Medium
FR-14	Order submission (future)	The system shall store orders in the backend in a future phase.	Medium
FR-15	Admin management (future)	The system shall provide admin features for categories, products, and orders in a future phase.	Low

5. External Interface Requirements

5.1 User Interface

- The homepage shall present categories and entry points to shopping flows.
- The category page shall use a responsive product grid and a search input.
- The product details page shall emphasize the selected product and related items.
- The cart page shall provide quantity controls and a clear pricing summary.
- The checkout page shall be structured as a simple, readable form with an order summary panel.
- Arabic pages shall render using RTL alignment, while English pages shall render using LTR alignment.

5.2 Software Interfaces

- Current phase: internal services layer for mock product/category retrieval.
- Future phase: REST API endpoints for categories, products, authentication, orders, and admin operations.
- Future phase: payment gateway integration subject to approved provider and business rules.

5.3 Hardware and Communication Interfaces

No specialized hardware interface is required. The application requires an internet-connected device capable of running a modern web browser.

6. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	Primary pages should render in under 3 seconds on a normal internet connection.
NFR-02	Responsiveness	The interface shall support mobile, tablet, and desktop layouts.
NFR-03	Usability	The storefront shall remain intuitive and easy to navigate within five or fewer main user actions from browsing to checkout.
NFR-04	Localization	The system shall support Arabic (RTL) and English (LTR) consistently.
NFR-05	Reliability	The UI shall handle missing data and invalid routes gracefully.
NFR-06	Maintainability	Code shall be organized into clear folders for pages, components, data, services, context, and localization.
NFR-07	Reusability	Common UI elements such as ProductCard shall be reusable across multiple screens.
NFR-08	Security	Sensitive credentials and secrets shall be stored in environment variables once the backend is introduced.
NFR-09	Scalability	The architecture shall allow future migration from mock data to API-backed persistence with limited refactoring.
NFR-10	Compatibility	The system shall work in the latest major desktop browsers.

7. Use Cases

UC ID	Use Case	Actor	Pre-condition	Post-condition
UC-01	Browse categories	Guest	User opens homepage	Category listing is visible
UC-02	Open category	Guest	User selects a category	Relevant products are shown
UC-03	Search products	Guest	User is on category page	Matching products are filtered
UC-04	View product details	Guest	User selects a product	Detailed product view is shown
UC-05	Add to cart	Guest	Product is available	Cart is updated
UC-06	Manage cart	Guest	Cart contains items	Quantities or items are updated
UC-07	Proceed to checkout	Guest	Cart is not empty	Checkout form is shown
UC-08	Submit order (future)	Registered User	Backend exists and checkout is valid	Order is stored and confirmation returned

8. Data Requirements

The current phase uses mock data, but the data model is designed to transition smoothly to SQLite in the backend phase.

8.1 Core Entities

Category entity

Field	Type	Description
id	Integer / String	Unique category identifier
slug	String	URL-friendly category key
nameAr	String	Arabic category name
nameEn	String	English category name
descriptionAr	String	Arabic description
descriptionEn	String	English description
image	String	Placeholder image path or future uploaded image

Product entity

Field	Type	Description
id	Integer / String	Unique product identifier
categoryId	Integer / String	Reference to category
slug	String	URL-friendly product key
nameAr	String	Arabic product name
nameEn	String	English product name
shortDescriptionAr	String	Arabic short description
shortDescriptionEn	String	English short description
descriptionAr	Text	Arabic detailed description
descriptionEn	Text	English detailed description
price	Decimal	Displayed product price
currency	String	Currency code or display symbol
image	String	Primary image path or placeholder
gallery	Array	Additional image paths if needed
inStock	Boolean	Availability state
featured	Boolean	Marks promoted products

Cart item entity

Field	Type	Description
productId	Integer / String	Selected product reference
quantity	Integer	Number of units selected
selectedOption	String	Optional variant or denomination
unitPrice	Decimal	Price at time of cart addition

Order entity (future)

Field	Type	Description
id	Integer / String	Unique order identifier
customerInfo	Object	Name, phone, email, and address details
items	Array	Ordered products and quantities
subtotal	Decimal	Total before delivery/fees
deliveryFee	Decimal	Applied delivery amount if used
total	Decimal	Final order total
paymentMethod	String	Selected payment option
status	String	Pending, processing, completed, etc.

9. Constraints, Assumptions, and Dependencies

9.1 Technical Constraints

- Frontend must remain based on React + Vite + Tailwind CSS.
- Current phase must remain frontend-only with mock data and local state.
- Backend introduction should target Express + SQLite.
- UI changes in refinement should not redesign the current visual identity.

9.2 Assumptions

- Users have access to a modern browser and stable internet connection.
- Placeholder images are acceptable during the development phase.
- Authentication, payment processing, and admin features are outside the current delivery scope.
- The bilingual structure will remain part of the final product vision.

9.3 Dependencies

Dependency	Purpose
React	Frontend component rendering
Vite	Development server and build pipeline
Tailwind CSS	UI styling
React Router	Page routing
Context API	Shared cart and language state
SQLite (future)	Relational persistence
Express (future)	Backend API layer

10. Agile Development Plan

10.1 Team Roles

Role	Responsibility	Who ?
Product / Project Owner	Defines scope, validates requirements, and prioritizes backlog	Hamed Bizerh
Scrum Master	Supports sprint planning, tracking, and delivery discipline	Baraa Masri
Frontend Development	Implements user-facing pages, components, cart flow, and UX polish	Collective ownership
Backend Development (future)	Will implement APIs, database, authentication, and order persistence	Collective ownership

10.2 Sprint Plan

Sprint	Focus	Expected Outcome
Sprint 1	Project setup, homepage, category/product/cart/checkout frontend flow	Stable storefront UI foundation
Sprint 2	Frontend stabilization, UX refinement, state cleanup, and edge-case handling	Polished frontend ready for backend integration
Sprint 3 (planned)	Backend APIs, SQLite persistence, authentication	nearly full-stack application
Sprint 4 (planned)	admin work, connect frontend with backend and deploy	Integrated full-stack application

10.3 Current Jira Direction

Completed early stories cover the storefront foundations such as ProductCard, CategoryPage, Cart page, and CartContext. The current refinement story focuses on code cleanup, state management consistency, and UX improvements before backend development begins.

11. Appendices

11.1 Planned API Summary (Future Phase)

Method	Endpoint	Purpose
GET	/api/categories	Return all categories
GET	/api/products	Return products, optionally filtered by category or search
GET	/api/products/:slug	Return single product details
POST	/api/auth/register	Create user account
POST	/api/auth/login	Authenticate user
POST	/api/orders	Submit a checkout order
GET	/api/orders/my	Return current user's orders
GET	/api/admin/orders	Return all orders for admin

11.2 Revision History

Version	Date	Summary of Changes
1.0	March 2026	Initial SRS draft focused on an earlier grocery/MERN concept.
2.0	April 2026	Revised to match the current YallaMarket digital marketplace scope, bilingual UI, frontend-first architecture, and future Express + SQLite plan.